

Curso de Go

Aula 22 - Projeto Final

Professor : Antônio Marcelo

NÚCLEA
ACADEMY

 **código_**^[3]
BRAZUCA





Nessa Aula nos Vamos Ver

- **Construiremos uma blockchain educacional em Go**
- **O projeto reunirá os principais conceitos vistos durante o curso**
- **Trabalharemos com structs, slices, funções, ponteiros, arquivos e JSON**
- **Utilizaremos SHA-256 para gerar os hashes dos blocos**
- **Implementaremos um mecanismo simples de Proof of Work**
- **Criaremos transações e blocos ligados por meio do PreviousHash**
- **A blockchain será salva em um arquivo ledger.json**
- **Ao reiniciar o programa, os blocos serão recuperados e a cadeia continuará do ponto onde parou**
- **Também implementaremos uma função para detectar alterações indevidas no ledger**
- **O objetivo é transformar vários conceitos de Go em uma aplicação prática e integrada**



Fundamentos de Go

- **Preparação do ambiente**
- **Structs**
- **Slices**
- **Ponteiros**
- **Funções**
- **Tratamento de erros**



Persistência de Dados

- **Trabalhando com arquivos**
- **Leitura e gravação com os**
- **JSON em Go**
- **Marshal e Unmarshal**
- **Criação do ledger.json**
- **Recuperação dos dados após reiniciar**



Fundamentos de Blockchain

- **Hash e SHA-256**
- **Estrutura de um bloco**
- **Genesis Block**
- **Previous Hash**
- **Encadeamento dos blocos**
- **Detecção de adulteração**



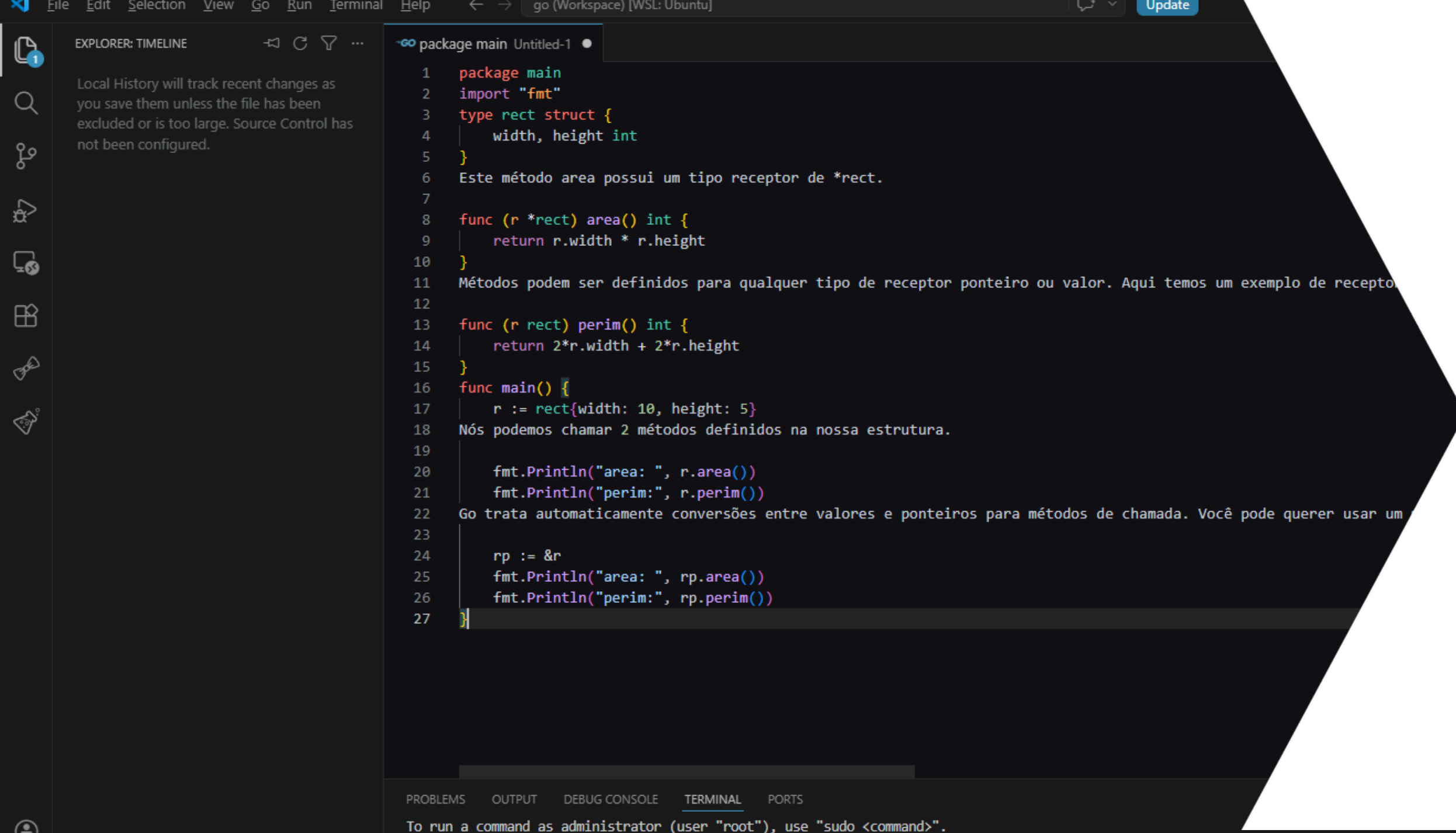
Mineração e Proof of Work

- **Conceito de Proof of Work**
- **Nonce**
- **Dificuldade**
- **Processo de mineração**
- **Número de tentativas**
- **Tempo de mineração**



Construção do Projeto Final

- Criação de transações
- Transações pendentes
- Mineração de novos blocos
- Validação da blockchain
- Persistência do ledger
- Menu da aplicação
- Testes de reinicialização e adulteração



The screenshot shows a Go IDE with a file named 'Untitled-1' in the 'package main' directory. The code defines a struct 'rect' with 'width' and 'height' of type 'int'. It includes two methods: 'area()' which returns the area, and 'perim()' which returns the perimeter. The 'main' function creates a 'rect' with width 10 and height 5, prints its area and perimeter, and then uses a pointer to the 'rect' to call the same methods, demonstrating pointer receivers. The IDE interface includes a sidebar with 'EXPLORER: TIMELINE' and a bottom panel with 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'.

```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de recepto
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

NÚCLEA
ACADEMY

Projeto Final

Código